

Article

Exploring the Impact of Battery Charge Reduction Rate and the Placement of Chargers on AGV Operation

Marius Gade ¹, Matthias Schedel ¹, Yuming Feng ²  and Reggie Davidrajuh ^{1,*} ¹ Department Electrical Engineering and Computer Science, University of Stavanger, 4021 Stavanger, Norway² School of Computer Science and Engineering, Chongqing Three Gorges University, Chongqing 404000, China; ymfeng@sanxiao.edu.cn

* Correspondence: reggie.davidrajuh@uis.no

Abstract: This paper presents a simulation model to study the effect of the battery charging rate of Automated Guided Vehicles (AGVs) on the overall output of a toy car production environment. This paper uses Modular Petri Nets for modeling and the General Petri Net Simulator (GPenSIM) for model implementation on MATLAB and simulation. The main focus of this paper is to analyze the operational efficiency of AGVs under varying conditions, such as the impact of battery charge reduction rates and the strategic placement of Charging Stations within the production line. By employing Modular Petri Nets implemented with GPenSIM, this paper presents a detailed model that captures the dynamics (movements and interactions) of AGVs in a simulated manufacturing environment. The model is also extensible, as newer functionalities can be added to it as Petri Modules. This paper specifically focuses on two critical operational parameters: (a) the number of AGVs and their battery charge reduction rate; (b) the number of Charging Stations. In summary, the goal, aim, and novelty of this paper is to provide a simpler yet effective model to practitioners so that they can study and experiment without needing advanced mathematical skills.

Keywords: automated guided vehicles; toy car assembly; Petri nets; GPenSIM; modeling and simulation; AGV battery life; AGV charging infrastructure; multi-sensor positioning; sensor networks



Citation: Gade, M.; Schedel, M.; Feng, Y.; Davidrajuh, R. Exploring the Impact of Battery Charge Reduction Rate and the Placement of Chargers on AGV Operation. *Machines* **2024**, *12*, 859. <https://doi.org/10.3390/machines12120859>

Academic Editor: Farshid Naseri

Received: 18 October 2024

Revised: 14 November 2024

Accepted: 22 November 2024

Published: 27 November 2024



Copyright: © 2024 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

The integration of automation and intelligent material handling systems has significantly accelerated the evolution of manufacturing processes. Among the most pivotal elements in modern production environments are Automated Guided Vehicles (AGVs), which have become indispensable in facilitating flexible and efficient transport of materials. This paper simulates AGVs in a toy car manufacturing scenario.

The simulation environment used for this paper is GPenSIM, an advanced toolbox developed for the MATLAB platform to model, simulate, and analyze discrete systems. GPenSIM provides a robust platform for simulating complex interactions and workflows characteristic of industrial processes.

The simulated production line comprises a series of interconnected stations, as shown in Figure 1: a warehouse for raw material storage, a CNC Machine for crafting the car's top half, a plastic molding station for the bottom part, a machine dedicated to tire manufacturing, another for creating the plastic windows, and an assembly machine that combines all the car components. Subsequent to assembly, the cars undergo painting and are then subjected to a quality control check to ensure that the final product meets the established standards.

AGVs, the backbone of our production line, are employed as the primary means of transportation between these stations, forming an integral part of the production line. AGVs that run out of battery charge get charged by one of five chargers. The objective of this paper is to provide a simulation platform so as to evaluate and compare the efficiency of the production line with a varying number of AGVs.

In this paper: Section 2 presents a compact literature study. Section 3 presents formal definitions of Petri nets and introduces GPenSIM. Section 4 presents the overall methodology for modeling a toy car production line as a Modular Petri Net model. The results of the analysis of the Modular Petri Net model, with a focus on AGV battery parameters, are presented in Section 5. Finally, Section 6 presents some limitations of this work.

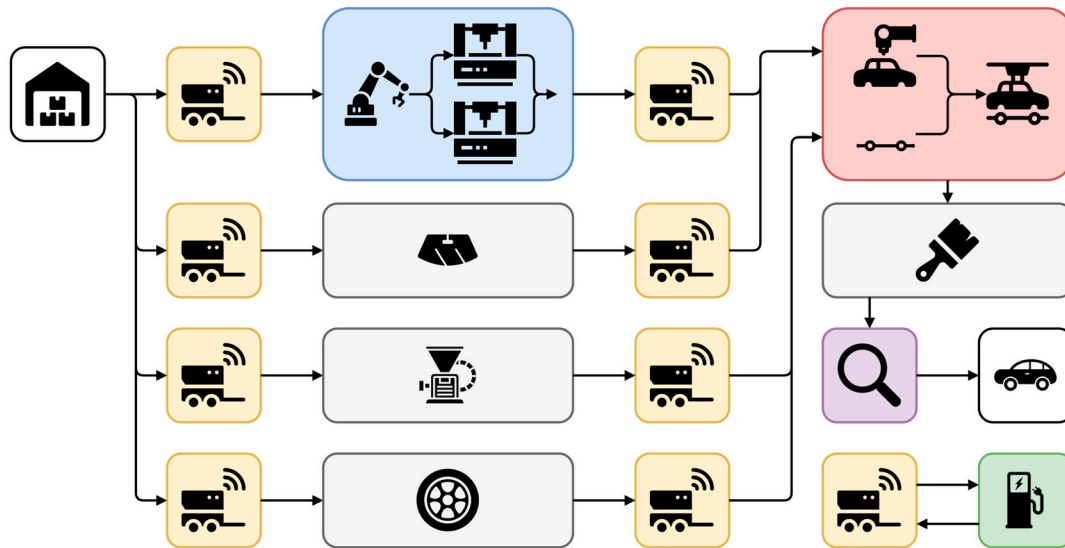


Figure 1. Schematic representation of the automated toy car production line with stations interconnected by AGV routes.

2. Relevant Works

The problem we are trying to solve in this paper is finding the impact of the battery charge reduction rate and the number and placement of Charging Stations on the production rate. A problem like this can be solved by:

1. Analytical methods (e.g., [1,2]).
2. Experimental works.
3. Simulation (for example, our work).

Analytical methods need a thorough understanding of the system dynamics and require high mathematical skills to draw equations that represent the problem; also, computation of the mathematical equations and the issues involved in the computation (such as stability, sensitivity, and convergence) require high mathematical skills. Also, analytical methods are not suitable when components of the system possess complex dependencies.

In simulations, as in our work, the model is developed to mimic the internal dynamics of the components and the dependency (the connections) between the components. Hence, both model development and computation do not require high mathematical skills. The key to model development is the observation of the dynamics involved in the systems. Hence, our work is novel, as the model is developed by observing the real physical system (a toy car assembly line), the model is componentized, and the components and their connections are modeled with Petri nets.

2.1. Analytical Works on AGV Charging

Ref. [3] studies the AGV scheduling problem with a focus on battery constraints. This work proposes a two-fold approach, one using a mixed integer linear programming for finding the bottlenecks, and the other a metaheuristic based on the sequential solution.

Ref. [4] studies AGVs in container terminals. This work considers the different power consumption of unloaded and loaded (empty) AGVs and the nonlinearity in the charging time of the batteries. This work also uses a metaheuristic algorithm based on neighborhood search.

Ref. [5] uses a mixed integer linear programming approach for finding optimal charging times. This work focuses on when and where to recharge an AGV when this AGV is already on an assignment, visiting a number of working stations and stops at Charging Stations.

With the aim of minimizing the tardiness costs of transport requests and travel costs of AGVs, Ref. [6] works on real-time dispatching of AGVs with a focus on battery constraints. This work formulates the real-time decision-making problem as a Markov Chain model and proposes a solution based on deep reinforcement learning.

Ref. [7] is also about AGVs in container terminals. This work proposes a speed control strategy to minimize the energy wasted when AGVs return to Charging Stations for recharging. This work uses a mixed integer programming model supplemented with a genetic algorithm-based approach for finding optimal initial conditions.

Ref. [8] focuses on AGVs in cyber-physical systems for fully autonomous operations. To improve AGV utilization and production efficiency, this work, however, proposes a Markov model for real-time charging of AGVs and a feature-based reinforcement learning approach for improving the utilization time.

Ref. [9] proposes a two-stage approach based on stochastic programming for the scheduling problem of battery swapping. This work emphasizes the dynamic changes and uncertainties in AGVs in container terminals. The first stage is to use constraints for battery swapping as a decision-making problem and solve it with stochastic programming. The second stage is to use a simulation-based ant colony optimization algorithm to improve the results.

2.2. Practical/Experimental Works on AGV Charging

Ref. [10] is on battery technology, focusing on Lithium-ion batteries as the primary energy storage devices used in AGVs. Lithium-ion batteries have higher energy density; hence, they are compact and provide longer cycle life. This work develops an AGV battery Charging Station specifically for charging lithium-ion batteries. This work uses the main charger (DPS5015), enhanced with an Arduino microcontroller for autonomous control. In addition, the Charging Station is equipped with the Internet of Things (IoT) to capture and store the charging data on a web base.

Ref. [11] proposes a practical approach using the commercial FlexSim software (version 7.3). This work focuses on AGV scheduling in the operation optimization of practical shopfloors with consideration of buffer space limitations and battery charging. This work uses a three rules-based approach: job selection, AGV selection, and Charging Station selection rules.

Ref. [12] takes a radically different look; this work looks into AGVs as an energy storage system so that AGVs can discharge electricity into a manufacturing facility when electricity demand is peak. Since the electricity pricing models penalize energy usage during peak demands, the concept of using AGVs as energy storage systems seems valid. This work uses a three-step approach: the first step is a calculation of energy consumption, the second step is the collection of the AGV operational data (routes, battery capacities, idle times, etc.), and the final step is decision making to determine when an AGV is to be used as energy storage or an energy consumer.

Ref. [13] studies the flexible charging of AGVs as an optimization problem to minimize battery charging. This work uses the commercial solver CPLEX to model the scenario as a mixed-integer linear programming model.

Ref. [14] looks into the wireless charging of AGVs versus conventional contact charging. This experimental work studies the volume of the power receiver side (50 Hz AC, low-voltage DC output for AGV batteries, voltage step-down transformers, and the types of existing solutions of AGV wireless charging). This work proposes a new architecture and a prototype for efficient wireless charging of AGVs.

2.3. Petri Net-Based Relevant Works

Ref. [15] proposes an intelligent Detection Method of Bottleneck (IDMOB) in discrete manufacturing systems, employing Object-Oriented Colored Petri Nets (OOCNPs) combined with cloud simulation. This approach abstracts production cells as Petri net sub-modules and uses cloud simulation to emulate real processing, demonstrating its effectiveness in a case study from the power transformer industry.

Ref. [16] uses the segmented flow topology, yet another approach to decompose Timed Petri Nets into smaller modules (servers in this work), to calculate the expected utilization of AGVs.

Ref. [17] proposes an approach using supervisory control (a formalization using a conjunction of linear constraints) when some of the transitions are uncontrollable. This work proposes the design of a maximally permissive controller using labeled Petri nets to prevent vehicles from any collision, in which labels represent signals generated by sensors.

Ref. [18] presents a control-based Petri net that translates a high-level AGV flow path into a low-level on/off control model. First, AGV flow paths are divided into smaller individual Petri Net (flow path) Modules, and then control modules are established using I/O control functions and station control macros. The flow path modules and control modules make up the complete AGV control Petri Net Module, avoiding the complexity of working on a holistic model. Ref. [19] focuses on the traffic control of AGVs in a large manufacturing system using Petri net models. A more macroscopic work on the motion control of AGVs is presented in [20].

Ref. [21] addresses the path planning problem of multi-type robot systems in industrial areas using Timed Colored Petri Nets. Their work focused on dividing tasks into various types and developing a planning approach for different types of mobile robots, including AGVs, to efficiently complete tasks within specified time windows. This study highlights the potential of Petri nets in optimizing the path planning and scheduling of AGVs in production environments. Refs. [22,23] propose approaches using new concepts known as “Related Places” and “Undirected Petri Nets”, respectively. Ref. [24] focuses on transportation systems for a semiconductor fabrication bay, as it requires collision-free route planning for multiple AGVs with a minimum total travel time. This work presents a Petri net-based approach for the optimization of route planning for multiple AGVs. The Petri net model is divided into many subnets, and the individual solutions of these subnets are put together to make the overall solution.

Ref. [25] focuses on the evaluation of reliability issues in AGV transportation of materials. In this work, the reliability of the AGV system is analyzed via Fault Tree Analysis (FTA) using a Petri net.

In [26], a Timed-Place Petri Net for a flexible manufacturing system with multiple AGVs that consists of two major submodels, namely a stationary transportation model and a variable process-flow model, primarily avoiding collisions and traffic jams. This work uses an A*-based search algorithm (known as a Limited-Expansion A algorithm) to search the paths of the firing sequence of Petri nets from the initial marking to the final marking, as the paths can be seen as schedules.

Ref. [27] focuses on the application of AGVs in container terminal operations using a Timed-Place Petri Net (TPPN). In this work, the Petri net model contains multiple modules, such as a layout module for a transportation road map and two submodules, such as an AGV submodule and a crane submodule. Ref. [28] studies the relationship between the invariant properties of Petri nets (such as safeness, boundedness, strict conservation, reachability, and liveness) and the properties of AGVs (collision-free traffic, numbers of AGVs and traffic control signals, path reachability, and deadlock-free traffic) so that an intelligent modelling tool can be developed to design robust AGV models. Ref. [29] also focuses on flexible manufacturing systems and the exploitation of real-time and multisource logistic data from manufacturing activities (termed the self-adaptive collaboration method). This work uses a Timed Colored Petri Net for the simulation in a manufacturing environment facilitated by Internet of Things technology (IoT) and Cloud computing.

Ref. [30] addresses the problem of managing and controlling AGVs on the manufacturing shop floor, focusing on the limited-capacity shared resources that are connected through a network of paths where collision is inevitable. This work uses a two-layer approach that puts a Petri net model on one layer (the Petri net represents the behavior of AGVs (such as the interactions among AGVs, path sections, and all other resources) and agents on the second layer (agents make decisions). Ref. [31] also uses supervisory control based on Colored Petri Nets; this work presents a zone control mechanism for collision avoidance and dynamic routing. This work uses undirected arcs and directed tokens to simplify the overall Petri net model and a semi-directed graph in which a graph can have both directed and undirected arcs.

The focus of our paper is unique. The literature study presented above shows that the works on modeling AGVs with Petri nets can be generally classified into the following categories:

- Componentization to reduce the complexity of the model.
- Control systems for AGVs.
- AGV path planning and routing.

The focus of our paper is different from the existing literature. In this paper, by varying the battery charge reduction rate, and the placement of chargers, we explore how different energy consumption models affect the overall productivity and efficiency of AGVs. This aspect is crucial in understanding the real-world implications of battery technology advancements on AGV operations. Furthermore, this paper paves an approach for the investigation into the optimal number of Charging Stations, which is a significant logistical challenge in AGV deployment.

3. Petri Nets and GPenSIM

This paper uses Petri nets to model the AGVs (more specifically, Modular Petri Nets).

3.1. Petri Nets

A Petri net is a bipartite graph that possesses two types of elements: (a) places and (b) transitions. Transitions represent active elements, such as events and actions. Places represent passive elements, such as buffers in a manufacturing facility or a hard disk of a computer. Arcs connect places and transitions. Being a bipartite graph, in a Petri net, an arc can only connect elements of different types (for example, an arc can connect a place to a transition and vice versa). Finally, tokens represent the material that flows through a network; for example, data packets in a computer network, or raw materials and semi-products in a manufacturing network.

Definition of Petri Nets

A (P/T) Petri net is a four-tuple [32,33]:

$$PTPN = (P, T, F, M_0), \quad (1)$$

where,

- P : set of places, $P = \{p_1, p_2, \dots, p_{np}\}$.
- T : set of transitions, $T = \{t_1, t_2, \dots, t_{nt}\}$.
 $P \cap T = \emptyset$.
- F : set of directed arcs; $F \subseteq (P \times T) \cup (T \times P)$. The default arc weight W of f_{ij} ($f_{ij} \in F$, an arc going from p_i to t_j or from t_i to p_j) is singleton, unless stated otherwise.
- M : row vector of markings (tokens) on the set of places.
 $M = [M(p_1), M(p_2), \dots, M(p_{np})] \in \mathbb{N}^{np}$, M_0 is the initial marking.

3.2. Modular Petri Nets

A Modular Petri Net consists of one or more Petri Modules, which are connected by zero or more Inter Modular Connectors (IMCs). A Petri Module is a Petri net that obeys some specific rules:

- Only through the Input Ports can tokens enter a Petri Module; Input Ports are transitions.
- Only through its Output Ports can tokens exit a Petri Module; Output Ports are also transitions.
- Local elements of a Petri Module (such as local places and local transitions) are not allowed to have direct connections (arcs) with outside elements (elements outside the Petri Module).

Using Modular Petri Nets (MPNs) provides two specific advantages [34]:

- Independent development of Petri Modules. Once the Petri Modules are thoroughly tested, we can use IMCs to connect the Petri Modules to create the whole model.
- Simulation of Petri nets takes a lot of time. Petri Modules, however, can run parallelly on different computers, reducing the simulation time.

Modular Petri Nets: Definition

Modular Petri Net (MPN) = Petri Modules + IMCs (Inter-Modular Connectors)

An MPN consists of one more Petri Module and zero or more IMCs.

Formal Definition of MPN:

An MPN is defined as a two-tuple:

$$MPN = (M, C)$$

- $M = \sum_{i=0}^m \Phi_i$ (one or more Petri Modules)
- $C = \sum_{i=0}^m \Psi_j$ (zero or more IMCs)

Formal Definition of Petri Module:

A **Petri Module** is defined as a six-tuple:

$$\Phi = (P_{L\Phi}, T_{IP\Phi}, T_{L\Phi}, T_{OP\Phi}, A_{\Phi}, M_{\Phi 0}),$$

where,

- $T_{IP\Phi} \subseteq T$: $T_{IP\Phi}$ (Input Ports)
- $T_{L\Phi} \subseteq T$: $T_{L\Phi}$ (local transitions)
- $T_{OP\Phi} \subseteq T$: $T_{OP\Phi}$ (Output Ports)
- $T_{IP\Phi}$, $T_{L\Phi}$, and $T_{OP\Phi}$, all are mutually exclusive:
 $T_{IP\Phi} \cap T_{L\Phi} = T_{L\Phi} \cap T_{OP\Phi} = T_{OP\Phi} \cap T_{IP\Phi} = \emptyset$.
- $T_{\Phi} = T_{IP\Phi} \cup T_{L\Phi} \cup T_{OP\Phi}$ (the transitions of the module).
- $P_{L\Phi} \subseteq P$ the local places. Since a module has only local places, $P_{\Phi} \equiv P_{L\Phi}$.
- $\forall p \in P_{L\Phi}$,
 - $\bullet p \in (T_{\Phi} \cup \emptyset)$. (a local place can be input by transition inside the module or none)
 - $p \bullet \in (T_{\Phi} \cup \emptyset)$. (an output of a local place can be transition inside the module or none)
- $\forall t \in T_{L\Phi}$,
 - $\bullet t \in (P_{L\Phi} \cup \emptyset)$. (input place of a module transition is a local place or none)
 - $t \bullet \in (P_{L\Phi} \cup \emptyset)$. (output place of a module transition is a local place or none)
- $\forall t \in T_{IP\Phi}$
 - $\bullet t \in (P_{L\Phi} \cup P_{IM} \cup \emptyset)$. (input places of Input Ports can be local places or places in IMCs or none)
 - $t \bullet \in (P_{L\Phi} \cup \emptyset)$. (output places of Output Ports can be local places or places in IMCs or none)
- $\forall t \in T_{OP\Phi}$

- $\bullet t \in (P_{L\Phi} \cup \emptyset)$. (input places of Output Ports can be local places or an empty set)
- $t\bullet \in (P_{L\Phi} \cup P_{IM} \cup \emptyset)$. (output places of Output Ports can be local places, IM places, or none)
- $A_\Phi \subseteq (P_L \times T_\Phi) \cup (T_\Phi \times P_L)$: where $a_{ij} \in A_\Phi$ (internal arcs)
- $M_{\Phi 0} = [M(p_L)]$ (initial markings in local places)

Formal Definition of Inter-Modular Connector:

An **Inter-Modular Connector (IMC)** is defined as a four-tuple:

$$\Psi = (P_\Psi, T_\Psi, A_\Psi, M_{\Psi 0})$$

where,

- $P_\Psi \subseteq P$: P_Ψ is the set of places in the IMC (known as the IM places). $\forall p \in P_\Psi$,
 - $\bullet p \in (T_{OP} \cup T_\Psi \cup \emptyset)$. (input transitions of IM places can be Output Ports of modules, IM transitions of this specific IMC, or none)
 - $p\bullet \in (T_{IP} \cup T_\Psi \cup \emptyset)$. (output transitions of IM places can be Input Ports of modules, IM transitions of this specific IMC, or none)
 (IM places are not allowed to have direct connections with local transitions of modules)
- $\forall p \in P_\Psi, \forall i p \notin P_{\Phi i}$ (a local place of a module cannot be an IM place).
- $T_\Psi \subseteq T$: T_Ψ represent the transitions of the IMC (*aka* IM transitions). $\forall t \in T_\Psi$,
 - $\bullet t \in (P_\Psi \cup \emptyset)$. (input places of IM transitions can be IM places of this specific IMC or none)
 - $t\bullet \in (P_\Psi \cup \emptyset)$. (output places of IM transitions can be IM places of this specific IMC or none)
- $\forall t \in T_\Psi, \forall i t \notin T_{\Phi i}$ (an IM transition is not allowed to be a member of any modules).
- $A_\Psi \subseteq (P_\Psi \times (T_\Psi \cup T_{IP})) \cup ((T_\Psi \cup T_{OP}) \times P_\Psi)$: where $a_{ij} \in A_\Psi$ is the IMC arc.
- $M_{\Psi 0} = [M(p_\Psi)]$ initial markings in IM places.

3.3. Why Modular Petri Nets

This paper uses Petri nets as the system under study (toy car assembly), which is fundamentally discrete. Though some other modeling formalisms like Markov Chains [35,36] and Automata [37,38] can be used to model discrete systems, we chose the Petri net because of its useful properties, such as a simple graphical front, well-established analytical tools, and self-documenting [39,40]. In this paper, we use Petri Modules because Petri Modules (Modular Petri Nets) allow independent development and testing of the components and easy assembly of the components together using the Inter-Modular Connectors. Also, Petri Modules can be run on different computers, thus minimizing the simulation time.

3.4. GPenSIM

GPenSIM is used in this paper to implement the Petri net model and for simulation. GPenSIM is an extended Petri net that uses pre-processor and post-processor files for coding the enabling functions [41]. Also, GPenSIM provides resources as an optional facility, which facilitates compact Petri net models for systems with a large number of resources.

GPenSIM runs on the MATLAB platform, and it was developed by the last author of this paper [41,42]. GPenSIM offers two major extensions that are used in this paper:

- Colored Petri Nets: this extension is useful for modeling real-life discrete systems [43].
- Modular Petri Nets: this extension is useful for modeling large discrete systems [34].

GPenSIM also provides a rich set of built-in functions to support modeling of discrete systems in any engineering domain [44].

4. Methodology

The toy car production line creates each part, combines them, and paints them before the product goes through a Fail Check to see if it functions properly and is safe for the public. Each of these processes is represented by a Petri Module in the Modular Petri Net (see Figure 2). For each token (representing material or product) to be able to move around the factory, we use AGVs. In the simulation, the movement of an AGV from point A to point B is also done in a separate Petri Module, as well as the charging of the AGVs. Hence, each process in the real-world scenario is represented by a Petri Module, following the rules.

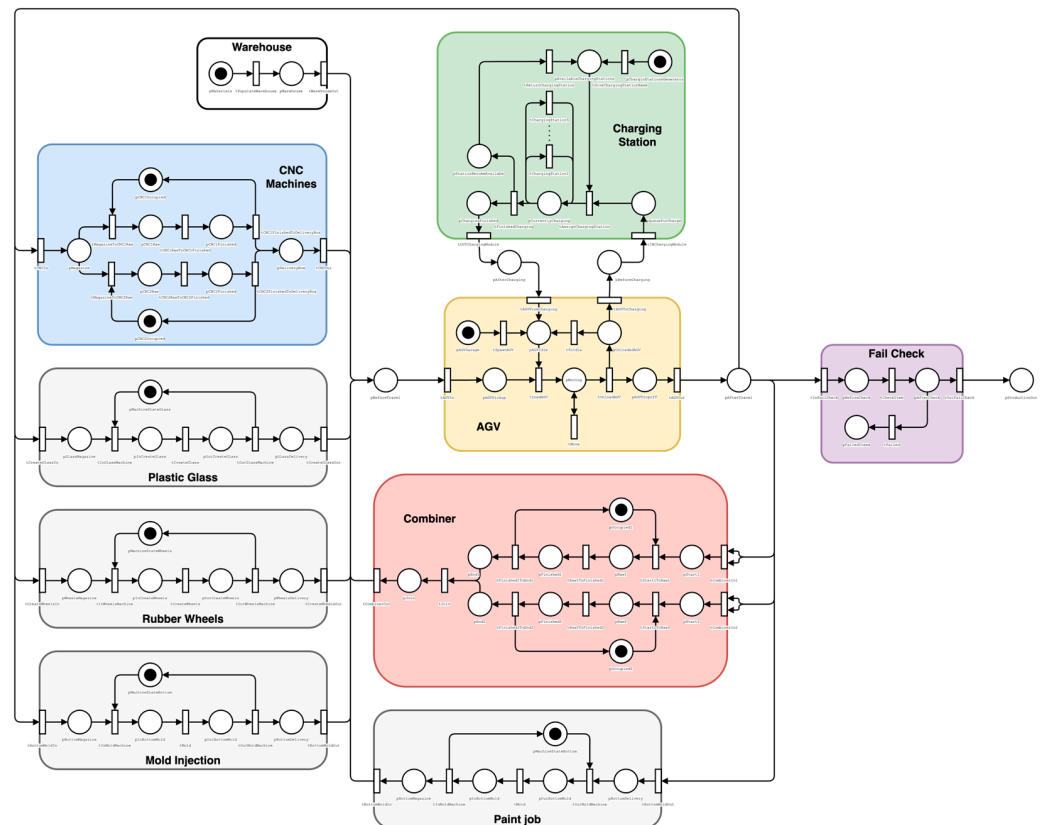


Figure 2. Modular Petri Net showing different Petri Modules used to simulate AGVs in the toy car manufacturing scenario. The internal details of the individual modules are explained in Figures 3–8.

4.1. Modular Approach

Dividing the whole Petri net into Petri Modules has several advantages over a non-modular approach: It allows us to implement each Petri Module individually with only minor adjustments needed to combine them into one large network later. It also makes the network more readable since different responsibilities are clearly divided. In the following sections, we will describe the Petri Modules in more detail, starting with the “Warehouse”, proceeding to the subsequent Modules in the production line, and ending with the “Fail Check”.

4.1.1. Warehouse

“Warehouse” is the simplest Petri Module; see Figure 3. This module is responsible for feeding a specified amount of materials into the network, which is achieved by first initializing pMaterials with 100 tokens. These tokens are then given colors by the tPopulateWarehouse transition. The transition will circle through a list of material names and assign them to the tokens. This assures that the materials in pWarehouse are distributed evenly. In the context of the complete Petri net shown in Figure 2, the importance of tWarehouseOut becomes apparent. This transition’s responsibility is to assign each token a destination

based on its material. The AGV Petri Module will later use this destination color to calculate the distance it has to travel with the token.

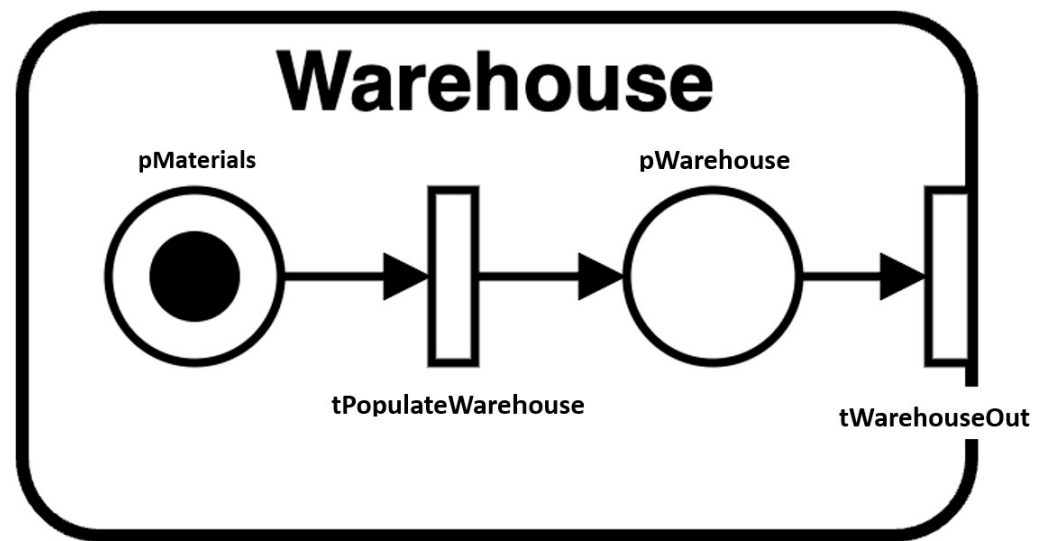


Figure 3. Petri Module representing the warehouse.

4.1.2. Automated Guided Vehicle

The AGV Petri Module plays a central role. This module is shown in Figure 4. When a token arrives in the pAFVPickup place, it will have the following attributes assigned as colors: type, location, destination. Based on this information, the tLoadAGV will calculate the distance the AGV will have to travel to transport the material to its destination. tLoadAGV will also assign one AGV token to the material. This is done so there is only a set number of AGVs traveling with materials between the simulated machines. The AGV tokens are initiated in the pAGVGarage place. In the transition tSpawnAGV, they are assigned a name, a value for the AGVs speed, and a battery charge of 100%. The AGV tokens will stay in pAGVIdle until they are needed to transport a material.

The place pMoving holds a number of tokens, which can be anywhere from 0 to the number of AGVs. They each have a color representing the AGVs traveled distance and its battery charge. These values are continuously updated by the tMove transition. Should an AGV run out of battery, it will no longer be able to move. Once an AGV's traveled distance is greater than the range it has to transport the material, the token will be taken out of pMoving by the transition tUnloadAGV. This transaction fires into two places: pAGVDropoff to represent the unloaded material and pUnloadedAGV to represent the empty AGV. The port tAGVOut has the important task of stripping the token of all colors that are associated with an AGV. It also removes the location at which the material started. This leaves a token with the following attributes embedded in its colors: material type, destination. Based on this information, the different modules representing the machines in the production line can later choose a token representing the correct material needed for their production sub-process.

The transitions tToIdle and tAGVToCharging check if the tokens' color representing the AGV's battery charge is greater or less than 20%. Should it be less than 20%, the token will be sent to the Charging Station module. If there is sufficient charge left in the AGV, it will be sent to pAGVIdle to wait for another travel. AGVs returning from the Charging Station will also end up in pAGVIdle. The observant reader will notice that this simulation omits the fact that an AGV will have to travel empty from the machine where it dropped off its material to the place where it will pick up the next material.

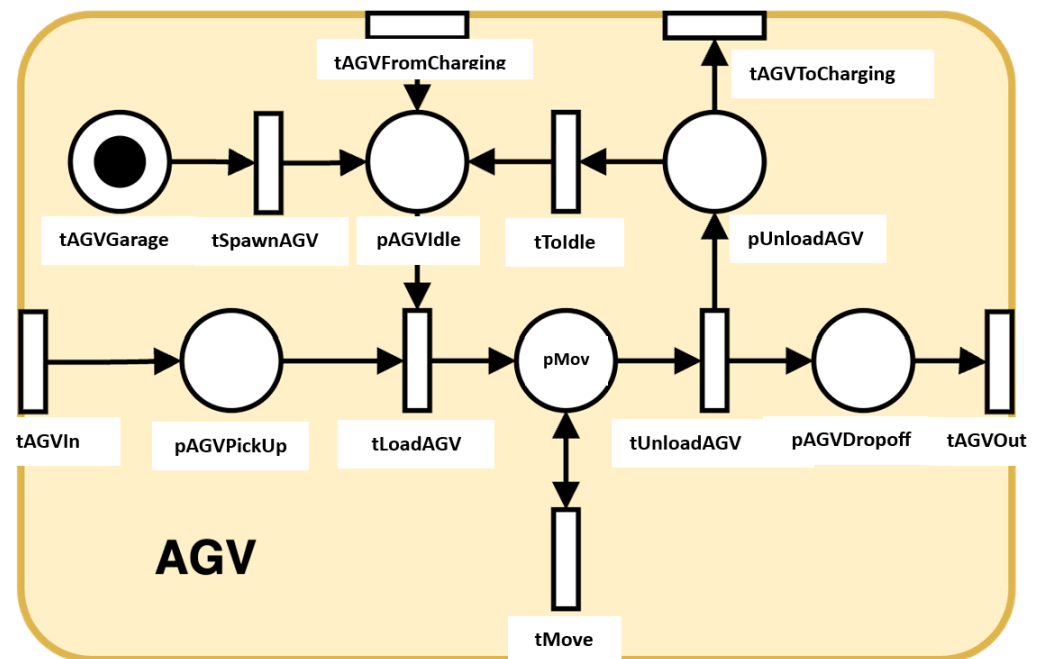


Figure 4. Petri Module representing the Automated Guided Vehicle (AGV).

4.1.3. Charging Station

The Charging Station is the Petri Module representing a set of Charging Stations that will charge the AGVs as they get low battery; see Figure 5. The transition *tINChargingModule* will work as the input port. *tINChargingModule* will only accept tokens with battery status 20% or below from *pBeforeCharging*. When an AGV gets into *pQueueForCharge*, it will either be assigned a Charging Station through adding of color in *tAssignChargingStation*, or it will await when a token gets available in *pAvailableChargingStations*. The Charging Stations gets the color they need by having uncolored tokens from *pChargingStationGenerator* go through the *tGiveChargingStationName*, where it will give the name 'ChargingStationX' where 'X' is the number one through the amount of Charging Stations in the module, which in our simulation is 5. The transition *tAssignChargingStation* will take a token from *pQueueForCharge* and a token from *pAvailableChargingStation*, where it will combine the color that is information about the AGV (name and battery) and the Charging Station color. When an AGV has a Charging Station, it will be placed in *pCurrentlyCharging*. It will then go through *tChargingStationX*, which will only accept tokens which have the 'ChargingStationX' color; this way, only one AGV can use a Charging Station at a time. Every time the token goes through *tChargingStationX*, it will take the colors, add (20%) to the battery color, and then put the colors back in the same order.

When the color that represents the battery in the token reaches 100, it cannot go any further up. When this happens, *tFinishedCharging* will take the token and place it in both *pStationBecomeAvailable* and *pChargingFinished*. The output *tOutPutChargingModule* will take a token from *pChargingFinished* and only keep everything except for the 'ChargingStationX' color. *tReturnChargingStation* does the exact opposite: it will take tokens from *pStationBecomeAvailable* and only take the 'ChargingStationX' color and put it in *pAvailableChargingStations*, so that 'ChargingStationX' can be used by another AGV.

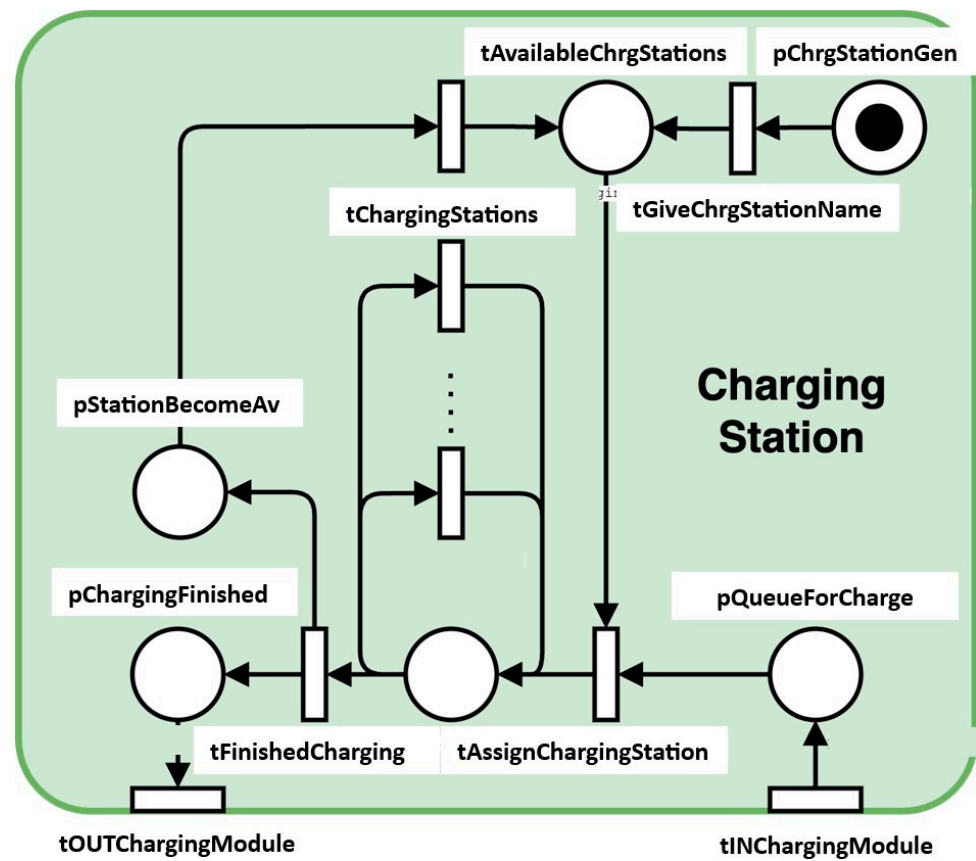


Figure 5. Petri Module representing the charging station.

4.1.4. CNC Machines

The CNC Machine is the Petri Module representing a set of two CNC Machines and one industrial robot arm. The robot arm moves raw material from a magazine to the CNC Machines, which can then work in parallel. The Petri Module representing this module is shown in Figure 6. In the transition $tCNCIn$, only tokens containing the color “Metal” will be taken in. They are stripped of all colors and placed in the $pMagazine$ place. From there, either $tMagazineToCNC1Raw$ or $tMagazineToCNC2Raw$ will take the token to pass it to $pCNC1Raw$ or $pCNC2Raw$, respectively. These transitions can only fire if they are able to take a token from their $pCNCxOccupied$ place, where ‘x’ represents the number of the CNC Machines, and if they can claim a resource representing the industrial robot. There is only one resource of type “Robot” available to simulate how only one CNC Machine can be loaded at a time. This resource is released as soon as the $tMagazineToCNCxRaw$ transition is done.

The transition $tCNCxRawToCNCxFinished$ will take a certain amount of time to process the raw material. Once the CNC operation is finished, the transition $tCNCxFinishedToDeliveryBox$ will release the CNC Machine by returning the token to the $pCNCxOccupied$ place and, using the industrial robot resource, the token representing the finished product will be placed in the $pDeliveryBox$ place.

Finally, the $tCNCOut$ returns the token to be picked up by an AGV. This means the type, location, and destination attributes have to be added as colors. To be specific, a token being fired by $tAGVOut$ will have the following values as colors: {‘upperBody’, ‘createTopHalf’, ‘combiner’}. Again, these colors are important for the AGV module to know how far the material has to travel to its destination.

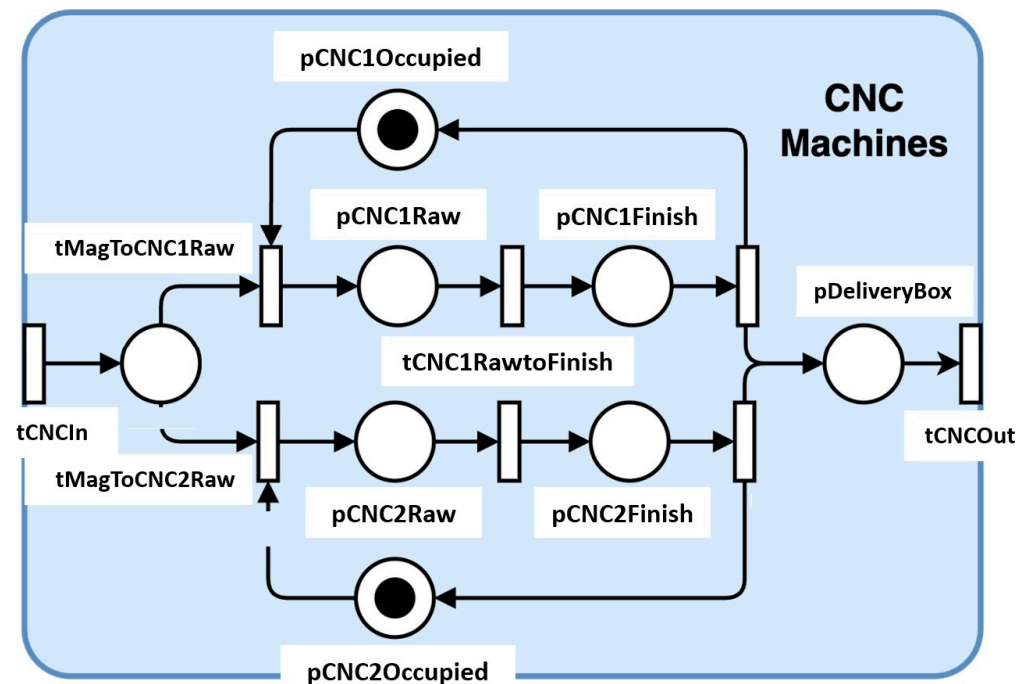


Figure 6. Petri Module representing the two CNC Machines working in parallel.

4.1.5. Plastic Glass, Rubber Wheels, Mold Injection, and Painting Job

The Petri Modules “Plastic Glass”, “Rubber Wheels”, “Mold Injection”, and “Paint Job” represent the construction or appliance of each of the items/jobs. Figure 7 shows the Petri Module for “Plastic Glass”; the other three Petri Modules are similar, possessing the same structure, as each of these jobs can be represented by a machine that takes a token, can only work on that token, and isn’t available until after it is finished with the token. We will use the Petri Module Plastic Glass (Figure 7) as an example to explain the mechanism.

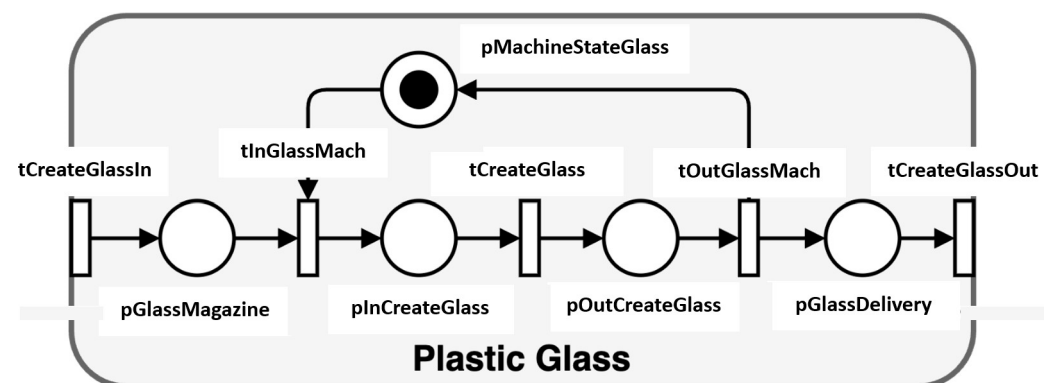


Figure 7. Petri net for the Plastic Glass module. This Petri net is the same for the Plastic Glass, Rubber Wheels, Mold Injection, and Paint Job modules.

Transition tCreateGlassIn works as the Input Port of the module. It will only accept tokens that have the ‘Plastic’ color from pAfterMat. The tokens with the color ‘Plastic’ will be put in pGlassMagazine. Transition tInGlassMachine will take a token from pGlassMagazine and from pMachineStateGlass and put one token in pInCreateGlass. Place pMachineStateGlass represents the availability of the machine; if there is no token, that means the machine is currently unavailable. This way, any tokens in pGlassMagazine must wait until there is a token in pMachineStateGlass before the transition tInGlassMachine can fire. Transition tCreateGlass represents the machine creating the plastic glass. When the transition is done, it will put the token in pOutCreateGlass, so pInCreateGlass and pOutCreateGlass

represents before the machine is done with the plastic glass and after it is done. Transition `tOutGlassMachine` will put a token in `pGlassDelivery` for the finished plastic glass and a token in `pMachineStatGlass` to show that the machine is available. `tCreateGlassOut` will take the token out of the module and give it the color `{'windshield', 'createPlacticGlass'}`, `'combiner'`. This will ensure the token is sent to the combiner module.

4.1.6. Combiner

The Petri Module Combiner represents an assembly machine that takes all the previously created parts like the upper body, the plastic glass, the tires, and the bottom part to combine them into an unpainted car.

The Combiner module has two Input Ports (`tCombiner1In` and `tCombiner2In`), one for the upper body and the glass and the other one for the lower body and the tires. They take two tokens each of which, for input 1, one has to have the color `"upperBody"` and one has to have the color `"windshield"`, and for input 2 the two tokens have to have the colors `"tires"` and `"lowerBody"`, respectively. The transition deposits only one token, representing both materials.

The two machines that do the work on the top and the bottom half are represented similarly to how the CNC Machines or the array of other processes were modeled in the previous section. `pStartx` represents the magazine before the machine starts working. To simulate a worker loading the two machines, transition `tStartxToRawx` requests the resource `'Combiner_Worker'`. Then again, similar to how the machines were implemented before, the machine takes the raw material from the `pRawx` places, works on them in the `tRawxToFinishedx` transition and gets unloaded from `pFinishedx`, again by the worker to `pEndx`. The occupation of a machine is simulated by initializing `pOccupiedx` with one token, which is returned when the machine is unloaded.

Then, the tokens from `pEnd1` and `pEnd2` are combined in the `tJoin` transition and placed in `pJoin`. Finally, `tCombinerOut` takes the token, gives it the colors `{'CarPutTogheter', 'combiner', 'painter'}` and moves it to the IMC place `pBeforeTravel` to be picked up by the AGVs.

4.1.7. Fail Check

This Petri Module, shown in Figure 8, represents the action of checking for errors in the final product. The Input Port `tInFailCheck` will only take tokens with the color `'finished_car'` from place `pAfterMat`. The tokens will then be placed in `pBeforeCheck`. As we don't have any actual way to check if the car 'works' properly in the simulation, the transition `tCheckItem` will just take a random number between 0 and 1, and if the number is over 0.8, the token will be assigned the color `'error'`. If the number is below 0.8, the token will get assigned the color `'passed'`. No matter which color it gets, the token will be placed `pAfterCheck`. From here, the tokens will either be taken by `tOutFailCheck` if they have the color `'passed'`, or taken by `tFailed` if they have the color `'error'`. This way, the output place of `tOutFailCheck` will contain all passed products, and `pFailedItems` will contain failed items. Hence, during the simulations and at the end of the simulation, we can then see how many tokens pass the test and how many tokens fail it. While this may not have any real value, it is mostly to simulate the fact that not every product will be shipped to the public; some products will fail the quality check and need to be thrown away or recycled.

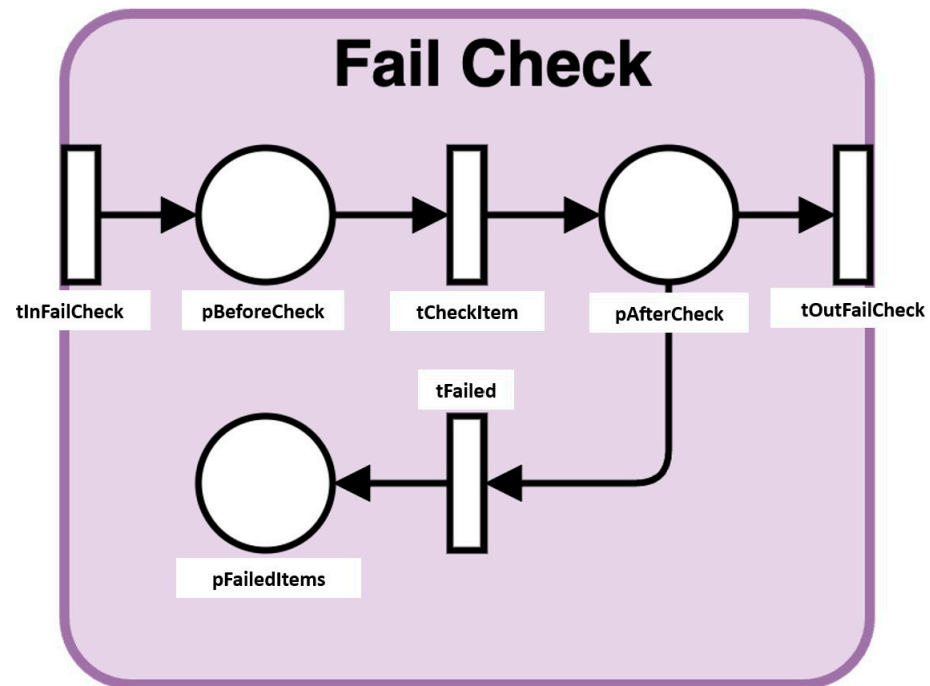


Figure 8. Petri Module representing Fail Check.

5. Testing, Analysis, and Results

The model implementation on the MATLAB (version R2024a) platform using GPenSIM resulted in many files. First, each Petri Module was implemented as a Petri Net Definition file. Also, for each module, there was a modular pre-processor file for implementing the additional logic conditions for firing enabled transitions. There were also some modular post-processor files for coding the post-firing actions.

Due to brevity, the implementation of the model (coding) is not presented in this paper. The complete code is available for any interested readers, along with a more detailed (technical) report and a user guide.

5.1. Sample Run

Figure 9 shows that the number of AGVs moving is mostly between six and eight. Figure 9 also shows that most of the time, there is only one AGV at the charging station, and rarely two AGVs. Also, nearly all the AGVs are active until the end of the simulation, in which more AGVs become idle one after the other.

In Figure 10, we can see the inventory at the warehouse, the number of products that passed the quality control, and how many failed over time. We can see that all materials are out of the warehouse at time step 100 and are being worked on in the production line. The first product is not finished until around 275–280 time steps, at which the first product gets approved. Not many time steps later, the first failed one arrives.

5.2. Studying the Impact of Battery

We simulated six runs using different combinations of the values for batteryConsumptionRate and the number of charging stations. We started with a baseline system where the AGVs had relatively robust batteries with a battery consumption rate of 0.5% per time unit and only four available charging stations. Then, we increased the battery consumption rate to 1% for every time unit, this time with five charging stations. Finally, we increased the battery consumption rate to 1.5% per time unit, which simulates a weaker AGV battery.

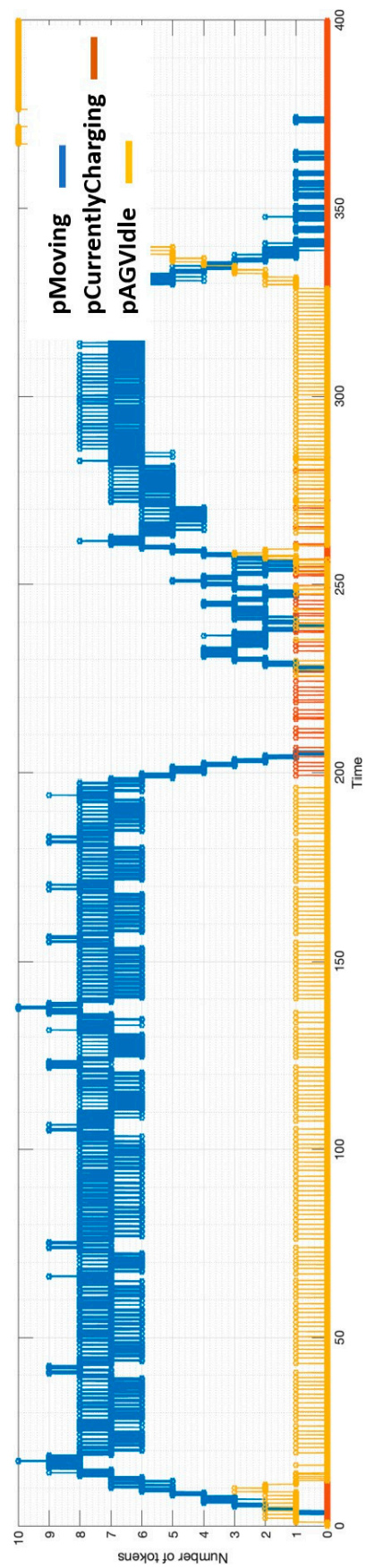


Figure 9. Status of the AGVs over time.

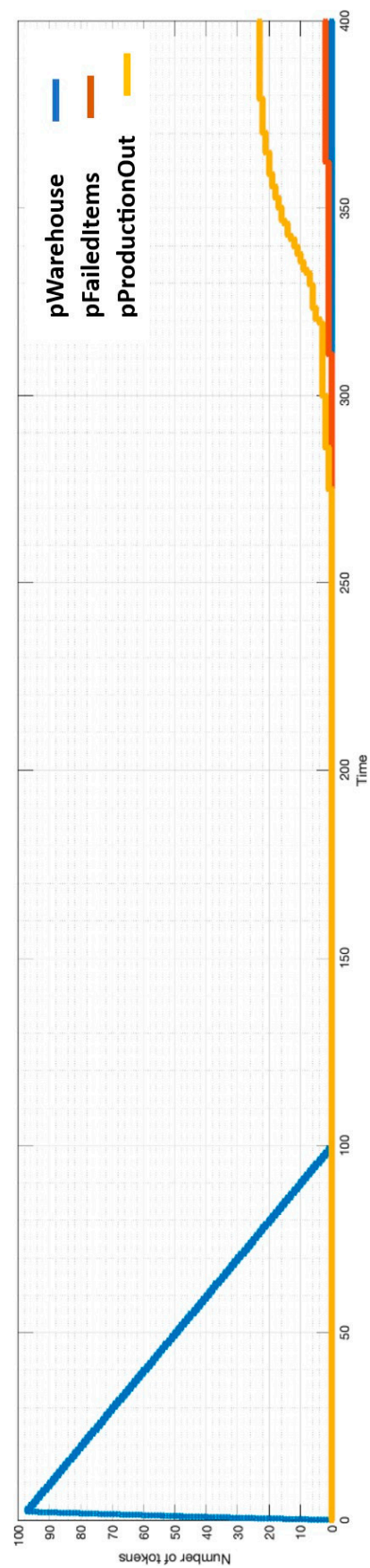


Figure 10. Number of different kinds of materials over time.

5.2.1. Case 2: 1% Battery Consumption Rate, Five Charging Stations

In Figure 11, we can see how the increased battery consumption rate leads to the need for the AGVs to be charged twice in order to transport all material. Comparing Figure 12 to the previous run with only 0.5% battery consumption rate, we can see that at time step 350, when the production in Case 1 had its maximum output per time unit, the production in Case 2 just starts to warm up.

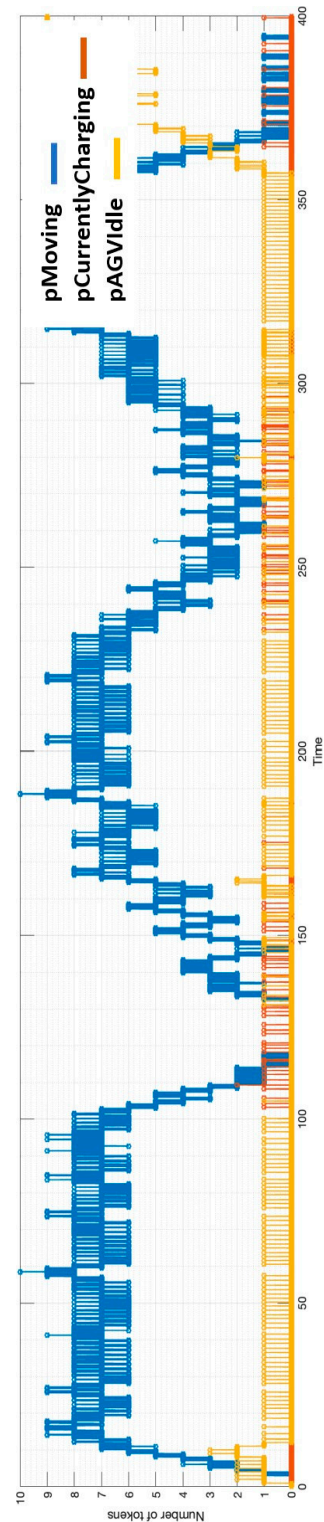


Figure 11. Case 2: Status of the AGVs over time.

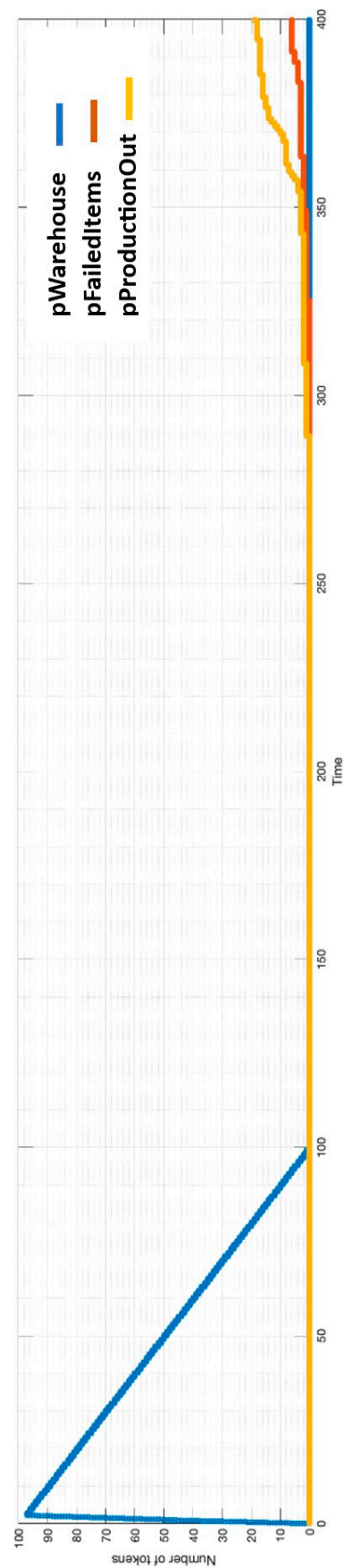


Figure 12. Case 2: Number of different kinds of materials over time.

5.2.2. Case 3: 1.5% Battery Consumption Rate, Five AGVs

Figure 13 shows how the increase in battery consumption rate to 1.5% causes the AGVs to be using a charging station almost constantly. This time, the 400 time steps of the simulation were not enough to turn all material tokens into the final product. Important to note is also that the production rate seems to have been lowered.

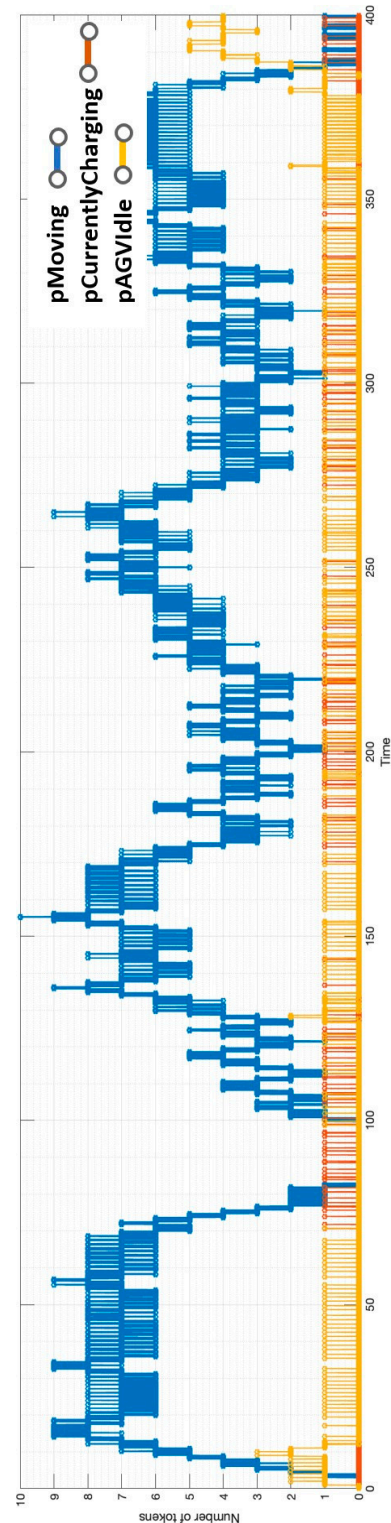


Figure 13. Case 3: Status of the AGVs over time.

Figure 14 clearly shows that increasing the battery consumption rate greatly impacts the system performance.

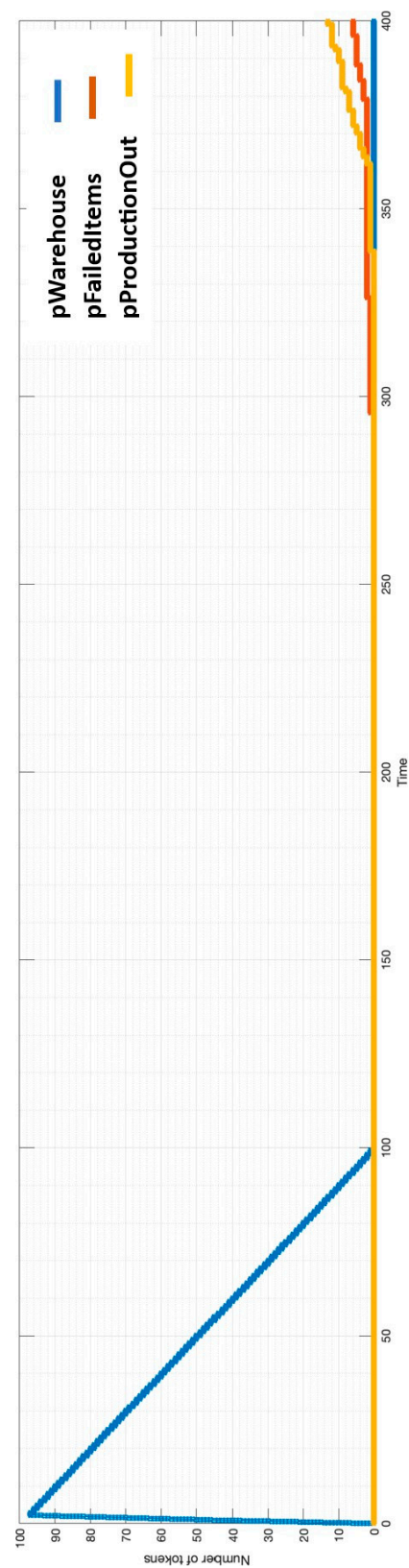


Figure 14. Case 3: Number of different kinds of materials over time.

5.2.3. Case 4: 2% Battery Consumption Rate, Four Charging Stations

In this case, we can see how the AGVs, displayed in Figure 15, run out of charge while moving and never reach their goal. The production stands still. No final product is produced, as shown in Figure 16.

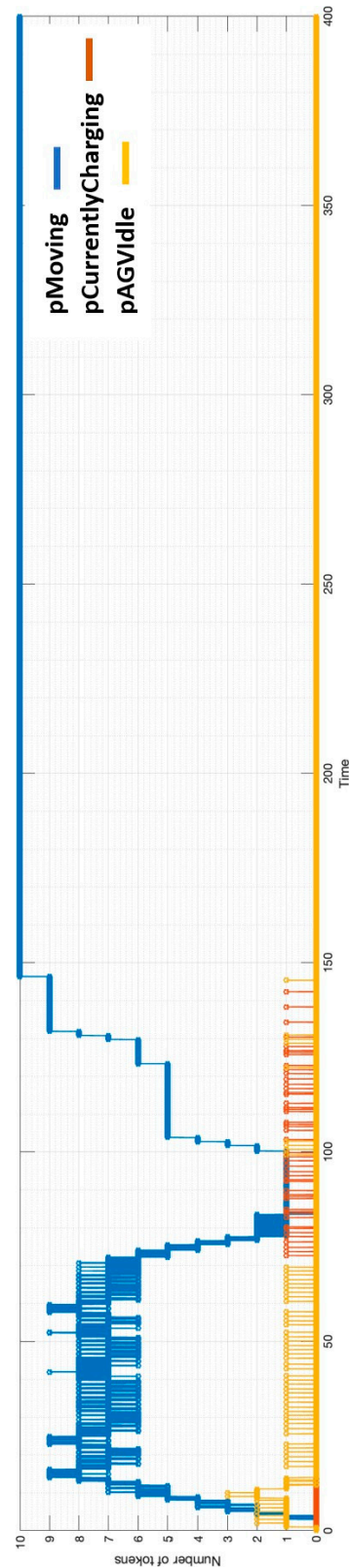


Figure 15. Case 4: Status of the AGVs over time.

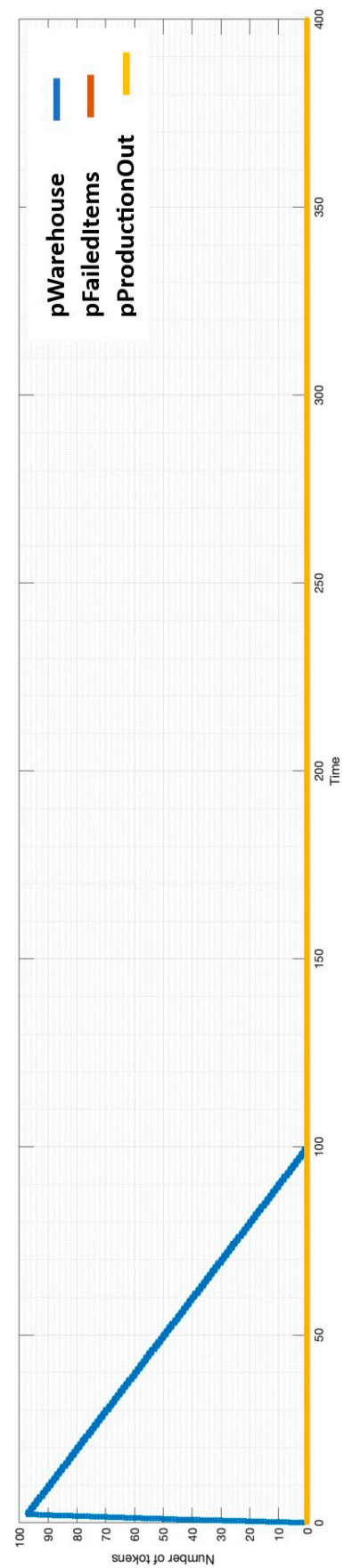


Figure 16. Case 4: Number of different kinds of materials over time.

The simulation framework allows for many more experimental setups, such as choosing a variable amount of AGVs or increasing the materials so that the production will continue for longer.

5.3. Summary of the Findings

We summarize the findings in all four cases in Table 1. In this table, **CS** stands for ‘the number of charging stations’ and **BCR** for ‘the battery consumption rate’.

Table 1. Summary of the findings in all four cases.

Case	Production Rate
Case 1: CS : 4, BCR : 0.5%	High (100 TU to empty the warehouse)
Case 2: CS : 5, BCR : 1%	Medium (350 TU to empty the warehouse)
Case 3: CS : 5, BCR : 1.5%	Slow (400 TU to empty the warehouse)
Case 4: CS : 5, BCR : 2.0%	Production stalled (complete stoppage)

6. Discussion

The novelty of the paper is that it uses Petri nets for a simple yet effective way of finding the impact of battery usage and the optimal number of AGVs in a real-life factory setting. The novelty of this paper lies in its approach to simulating AGVs in a production line environment specifically designed for the assembly of toy cars. This study stands out in its detailed focus on two critical operational parameters: the battery charge reduction rate of AGVs and the number of charging stations available in the production line.

Unlike previous studies that have primarily concentrated on general path planning or bottleneck detection in manufacturing systems, our research delves into the intricate balance between AGV operational efficiency and the logistical constraints imposed by battery life and charging infrastructure.

In the Petri net model, by varying the battery charge reduction rate, we explore how different energy consumption models affect the overall productivity and efficiency of AGVs. This aspect is crucial in understanding the real-world implications of battery technology advancements on AGV operations. Furthermore, the investigation into the optimal number of charging stations addresses a significant logistical challenge in AGV deployment, providing insights into how production lines can be designed or modified to maximize AGV uptime and throughput.

Additionally, the use of a toy car production line as a test bed adds a unique dimension to our study. This controlled yet realistic environment allows for a detailed examination of AGV dynamics in a manufacturing setting, offering valuable data that can be extrapolated to larger, more complex production systems. The findings from this study have the potential to inform future designs and operational strategies for AGV-integrated production lines, particularly in industries where precision and efficiency are important.

Limitations of This Work

While our study provides valuable insights into the simulation of AGVs in a production environment, it is important to acknowledge certain limitations. Firstly, our model does not account for the distance traveled by an AGV when it moves between different points in the production line. In the current setup, AGVs are assumed to instantly appear at the required location without consuming battery power or covering any actual distance. This simplification overlooks the potential impact of travel distance on battery consumption and overall efficiency.

Another notable limitation is the absence of a director module in our current framework. Such a module could significantly enhance the operational dynamics by managing the stock of materials and semi-finished products and directing AGVs for efficient resource allocation and transportation within the production environment.

Furthermore, our Petri net model primarily focuses on a specific segment of the manufacturing process. Extending this model to encompass other stages, such as shipping, could provide a more comprehensive view of the entire product life cycle, from creation to sale.

7. Conclusions

As can be seen in Section 2, there are numerous works on AGVs and battery charging. Most of the works are analytical, using mixed integer programming, stochastic programming, simulated annealing, etc. Most of these works also use a two-step approach in which the first step is to model and analyze the problem as an optimization problem, and then the second step is to fine-tune the results using deep learning techniques. It is noteworthy that these works require a high level of mathematical skills. Literature studies reveal that some more works are technical or experimental, in which experiments find the optimal battery charging.

Our paper uses a Petri net model, which is easy to study and understand; also, our model is easy to adapt and expand, as newer functionality can be incorporated by adding newer modules into the modular model. In summary, the goal, aim, and novelty of this paper is to provide a simpler yet effective model to practitioners so that they can study and experiment without needing advanced mathematical skills. Also, the focus of our paper, namely the effect of varying the battery charge reduction rate on the overall productivity and efficiency of AGVs, is a realistic problem faced by manufacturing facilities. This aspect is crucial in understanding the real-world implications of battery technology advancements on AGV operations. Furthermore, this paper paves an approach for the investigation into the optimal number of charging stations, which is a significant logistical challenge in AGV deployment.

Author Contributions: Conceptualization, M.G. and M.S.; methodology, M.G., M.S. and R.D.; software, M.G., M.S., Y.F. and R.D.; validation, M.G. and M.S.; formal analysis, M.G., M.S. and Y.F.; investigation, M.G. and M.S.; resources, M.G., M.S., Y.F. and R.D.; writing—original draft preparation, M.G., M.S. and R.D.; writing—review and editing, Y.F.; visualization, M.G. and M.S.; supervision, R.D.; project administration, Y.F. and R.D. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Data Availability Statement: Data are contained within the article.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Zhan, X.; Xu, L.; Zhang, J.; Li, A. Study on AGVs battery charging strategy for improving utilization. *Procedia CIRP* **2019**, *81*, 558–563. [\[CrossRef\]](#)
2. Abderrahim, M.; Bekrar, A.; Trentesaux, D.; Aissani, N.; Bouamrane, K. Manufacturing 4.0 operations scheduling with AGV battery management constraints. *Energies* **2020**, *13*, 4948. [\[CrossRef\]](#)
3. Boccia, M.; Masone, A.; Sterle, C.; Murino, T. The parallel AGV scheduling problem with battery constraints: A new formulation and a matheuristic approach. *Eur. J. Oper. Res.* **2023**, *307*, 590–603. [\[CrossRef\]](#)
4. Song, X.; Chen, N.; Zhao, M.; Wu, Q.; Liao, Q.; Ye, J. Novel AGV resilient scheduling for automated container terminals considering charging strategy. *Ocean Coast. Manag.* **2024**, *250*, 107014. [\[CrossRef\]](#)
5. Iudice, F.L.; Olm, J.M.; Suárez, R. Optimal recharge sequencing in multi-AGV systems: A mixed ILP approach. *Comput. Ind. Eng.* **2024**, *196*, 110510. [\[CrossRef\]](#)
6. Singh, N.; Akcay, A.; Dang, Q.V.; Martagan, T.; Adan, I. Dispatching AGVs with battery constraints using deep reinforcement learning. *Comput. Ind. Eng.* **2024**, *187*, 109678. [\[CrossRef\]](#)
7. Yang, X.; Hu, H.; Cheng, C.; Wang, Y. Automated guided vehicle (agv) scheduling in automated container terminals (acts) focusing on battery swapping and speed control. *J. Mar. Sci. Eng.* **2023**, *11*, 1852. [\[CrossRef\]](#)
8. Lee, M.S.; Hwang, I.; Jang, S.; Im, N.J.; Jang, Y.J. Markov Decision Process Approach for Battery Charging of an Automated Guided Vehicle. In Proceedings of the 2024 21st International Conference on Ubiquitous Robots (UR), New York, NY, USA, 24–27 June 2024; pp. 138–145.

9. Li, L.; Li, Y.; Liu, R.; Zhou, Y.; Pan, E. A Two-stage Stochastic Programming for AGV scheduling with random tasks and battery swapping in automated container terminals. *Transp. Res. Part E Logist. Transp. Rev.* **2023**, *174*, 103110. [\[CrossRef\]](#)
10. Tan, T.W. Development of Battery Charging Station for Automated Guided Vehicle. Ph.D. Thesis, UTAR, Kampar, Malaysia, 2023.
11. Park, J.S.; Kim, J.W. Multi-AGV Scheduling under Limited Buffer Capacity and Battery Charging Using Simulation Techniques. *Appl. Sci.* **2024**, *14*, 1197. [\[CrossRef\]](#)
12. Yesilyurt, O. Approach Development for Enabling Energy Flexibility in Manufacturing Companies with AGV Lithium Batteries Through an Algorithm-Based ICT Solution. Ph.D. Thesis, Fraunhofer-Institut für Produktionstechnik und Automatisierung IPA, Stuttgart, Germany, 2024.
13. Li, Y.; Ren, B.; Liu, L.; Tang, L. AGV Scheduling Problem with Flexible Charging Operations and Limited Number of Charging Stations. In Proceedings of the 2023 IEEE International Conference on Networking, Sensing and Control (ICNSC), Marseille, France, 25–27 October 2023; Volume 1, pp. 1–6.
14. Liao, Y.; Jiang, Q.; Guo, D. Wireless Charging for AGV: An Analysis to the Existing Solutions and a Novel Architecture. In Proceedings of the International Conference on Wireless Power Transfer; Springer: Singapore, 2023; pp. 437–444.
15. Zhang, Y.; Peng, E.; Luo, Z.; Zeng, L.; Li, C. Bottleneck detection for discrete manufacturing system based on object-oriented colored petri nets and cloud simulation. *ITM Web Conf.* **2022**, *45*, 01015. [\[CrossRef\]](#)
16. Barad, M.; Sinriech, D. A Petri net model for the operational design and analysis of segmented flow topology (SFT) AGV systems. *Int. J. Prod. Res.* **1998**, *36*, 1401–1425. [\[CrossRef\]](#)
17. Luo, J.; Wan, Y.; Wu, W.; Li, Z. Optimal Petri-net controller for avoiding collisions in a class of automated guided vehicle systems. *IEEE Trans. Intell. Transp. Syst.* **2019**, *21*, 4526–4537. [\[CrossRef\]](#)
18. Hsieh, S.; Kang, M.Y. Developing AGVS Petri net control models from flowpath nets. *J. Manuf. Syst.* **1998**, *17*, 237–250. [\[CrossRef\]](#)
19. Shuhua, H.; Shieh, K.-H.; Lin, M. On Some Basic AGV Traffic Control Structures Modeled by Petri Nets. *Eng. J. Natl. Taiwan Univ.* **1993**, 113–124. [\[CrossRef\]](#)
20. Hernandez-Martinez, E.G.; Foyo-Valdes, S.A.; Puga-Velazquez, E.S.; Campana, J.A.M. Motion Coordination of AGV's in FMS using Petri Nets. *IFAC-PapersOnLine* **2015**, *48*, 187–192. [\[CrossRef\]](#)
21. He, Z.; Zhang, R.; Ran, N.; Gu, C. Path Planning of Multi-Type Robot Systems with Time Windows Based on Timed Colored Petri Nets. *Appl. Sci.* **2022**, *12*, 6878. [\[CrossRef\]](#)
22. Zhang, H.; Fang, Z.; Fu, W.; Wang, H. AGV path planning method based on Petri net. *Comb. Mach. Tools Autom. Mach. Technol.* **2001**, *2008*, 21–24.
23. Ren, X.; Wen, H.; Li, H. Research on multi-AGV optimal path method for undirected Petri nets. *J. Xidian Univ.* **2008**, *35*, 517–522.
24. Nishi, T.; Maeno, R. Petri net decomposition approach to optimization of route planning problems for AGV systems. *IEEE Trans. Autom. Sci. Eng.* **2010**, *7*, 523–537. [\[CrossRef\]](#)
25. Yan, R.; Jackson, L.M.; Dunnett, S.J. Automated guided vehicle mission reliability modelling using a combined fault tree and Petri net approach. *Int. J. Adv. Manuf. Technol.* **2017**, *92*, 1825–1837. [\[CrossRef\]](#)
26. Sun, T.H.; Cheng, C.W.; Fu, L.C. A Petri net based approach to modeling and scheduling for an FMS and a case study. *IEEE Trans. Ind. Electron.* **1994**, *41*, 593–601.
27. Liu, C.I.; Ioannou, P.A. Petri net modeling and analysis of automated container terminal using automated guided vehicle systems. *Transp. Res. Rec.* **2002**, *1782*, 73–83. [\[CrossRef\]](#)
28. Hsieh, S.; Shih, Y.J. Automated guided vehicle systems and their Petri-net properties. *J. Intell. Manuf.* **1992**, *3*, 379–390. [\[CrossRef\]](#)
29. Guo, Z.; Zhang, Y.; Zhao, X.; Song, X. A timed colored petri net simulation-based self-adaptive collaboration method for production-logistics systems. *Appl. Sci.* **2017**, *7*, 235. [\[CrossRef\]](#)
30. Giglio, D.; Paolucci, M. Agent-based Petri net models for AGV management in manufacturing systems. In Proceedings of the 2001 IEEE International Conference on Systems, Man and Cybernetics. e-Systems and e-Man for Cybernetics in Cyberspace (Cat.No. 01CH37236), Tucson, AZ, USA, 7–10 October 2001; Volume 4, pp. 2457–2462.
31. Roszkowska, E. Undirected colored Petri net for modelling and supervisory control of AGV systems. In Proceedings of the Sixth International Workshop on Discrete Event Systems, Zaragoza, Spain, 4 October 2002; pp. 135–142.
32. Murata, T. Petri nets: Properties, analysis and applications. *Proc. IEEE* **1989**, *77*, 541–580. [\[CrossRef\]](#)
33. Peterson, J.L. *Petri Net Theory and the Modeling of Systems*; Prentice Hall PTR: Hoboken, NJ, USA, 1981.
34. Davidrajuh, R. *Petri Nets for Modeling of Large Discrete Systems*; Springer: Berlin/Heidelberg, Germany, 2021.
35. Hermanns, H.; Hermanns, H. Markov Chains. In *Interactive Markov Chains: And the Quest for Quantified Quality*; Springer: Berlin/Heidelberg, Germany, 2002; pp. 35–55.
36. Behrends, E. *Introduction to Markov Chains*; Springer: Wiesbaden, Germany, 2000; Volume 228.
37. Nelson, R.J. *Introduction to Automata*; Wiley: New York, NY, USA, 1968.
38. Khoussainov, B.; Nerode, A. *Automata Theory and Its Applications*; Springer Science & Business Media: Berlin/Heidelberg, Germany, 2012; Volume 21.
39. Deplomo, B.N.I.; Villaverde, J.F.; Paglinawan, A.C. Dynamic Simulation and Modeling of a Novel NeuRaiSys for Railway Monitoring System Using Petri Nets. *Sensors* **2024**, *24*, 4095. [\[CrossRef\]](#)
40. Han, Y.; Zhang, L.; Wang, Y.; Deng, X.; Gu, Z.; Zhang, X. Research on the Security of IPv6 Communication Based on Petri Net under IoT. *Sensors* **2023**, *23*, 5192. [\[CrossRef\]](#)

41. Davidrajuh, R. *Modeling Discrete-Event Systems with GPenSIM*; Springer International Publishing: Cham, Switzerland, 2018. [[CrossRef](#)]
42. GPenSIM. General-Purpose Petri Net Simulator. Technical Report. 2019. Available online: <http://www.davidrajuh.net/gpensim> (accessed on 20 July 2020).
43. Davidrajuh, R. *Colored Petri Nets for Modeling of Discrete Systems: A Practical Approach with GPenSIM*; Springer Nature: Singapore, 2023.
44. Davidrajuh, R. *GPenSIM Reference Manual*; University of Stavanger: Stavanger, Norway, 2018. [[CrossRef](#)]

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.